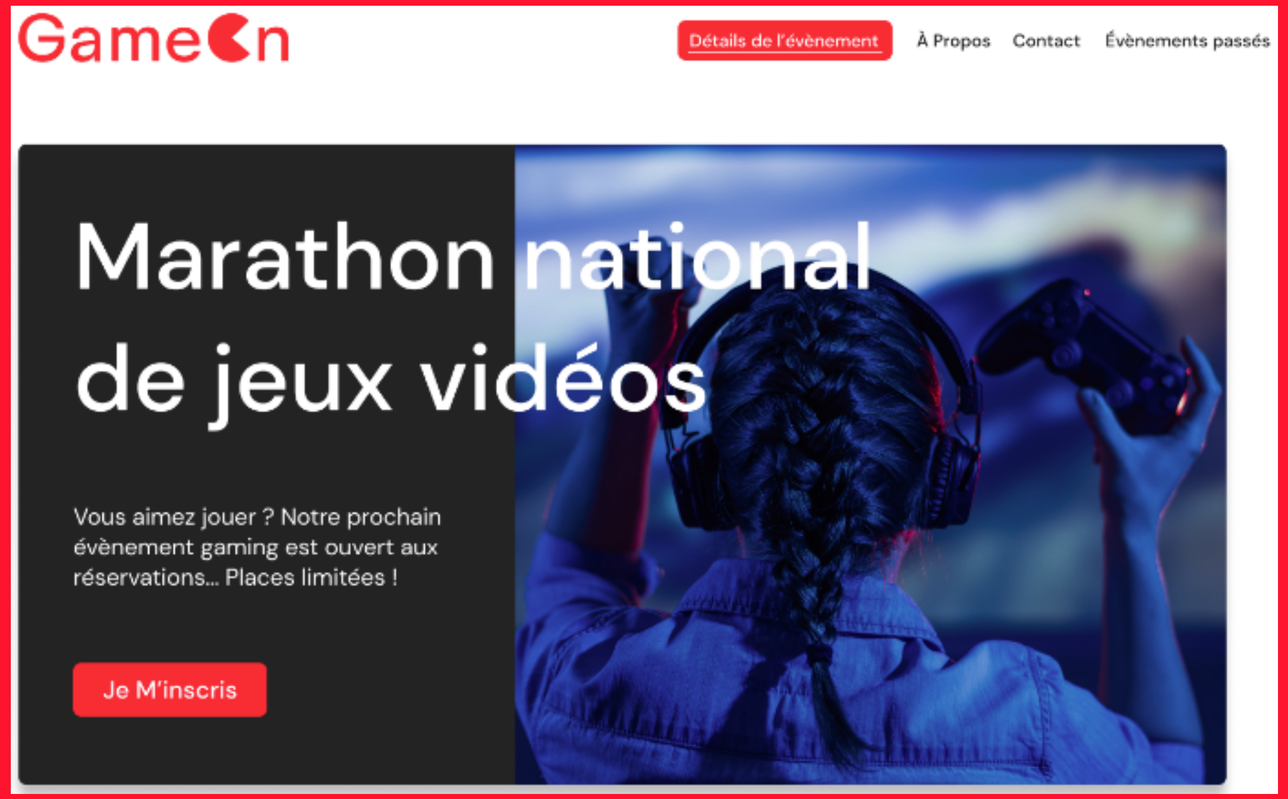
Soutenance Projet 4 OpenClassrooms

GameOn

Créez une landing page avec Javascript

Présenter par Ghani

PROJET 4



01

Présentation du projet

02

Code JavaScript

03

Tests manuels

Vous avez récemment été embauché comme développeur Front-End junior dans une PME, GameOn, spécialisée dans les conférences et les concours de jeux.
Vous avez fait du shadowing de votre développeur principal, Erika, pour suivre son travail au quotidien.

GameOn

Présentation du projet

Un matin, vous recevez le courriel suivant :

Bonjour,

Comme tu le sais, nous avons simplifié la nouvelle page d'accueil et l'inscription aux concours après avoir reçu des commentaires négatifs de la part de certains utilisateurs. Jason a travaillé dessus, mais malheureusement, il n'a pas eu le temps de terminer avant de partir pour son nouveau poste dans sa nouvelle entreprise. J'aurai besoin que tu prennes le relais et que tu complètes la partie Front-End.

Pour te donner un aperçu, Jason avait commencé à travailler sur la structure HTML et le style CSS, sur la base de ces maquettes. Il a terminé la mise en page et le contenu de la page d'accuei ainsi que de la modale du formulaire. Il a également ajouté le JavaScript pour le lancement de la maquette. Voici le lien vers le repo avec le code qu'il a déjà complété.

Ton travail consiste à ajouter le code JavaScript manquant pour que le formulaire soit pleinement fonctionnel. Pour t'aider à démarrer, j'ai demandé à Jason d'ajouter les issues sur le repo GitHub qui décrivent ce qu'il reste à faire.

Nous avons reçu quelques commentaires de la part de l'équipe de QA récemment, donc lorsque tu soumets ton code, assure-toi de :

- travailler sur un repo GitHub forké ;
- utiliser des fichiers séparés pour le HTML, le CSS et le JavaScript ;
- toujours commenter ton code (décrire chaque fonction et chaque classe, ainsi que les parties du code qui nécessitent plus de détails) ;
- tester manuellement les fonctionnalités, les entrées de formulaire et l'affichage responsive.

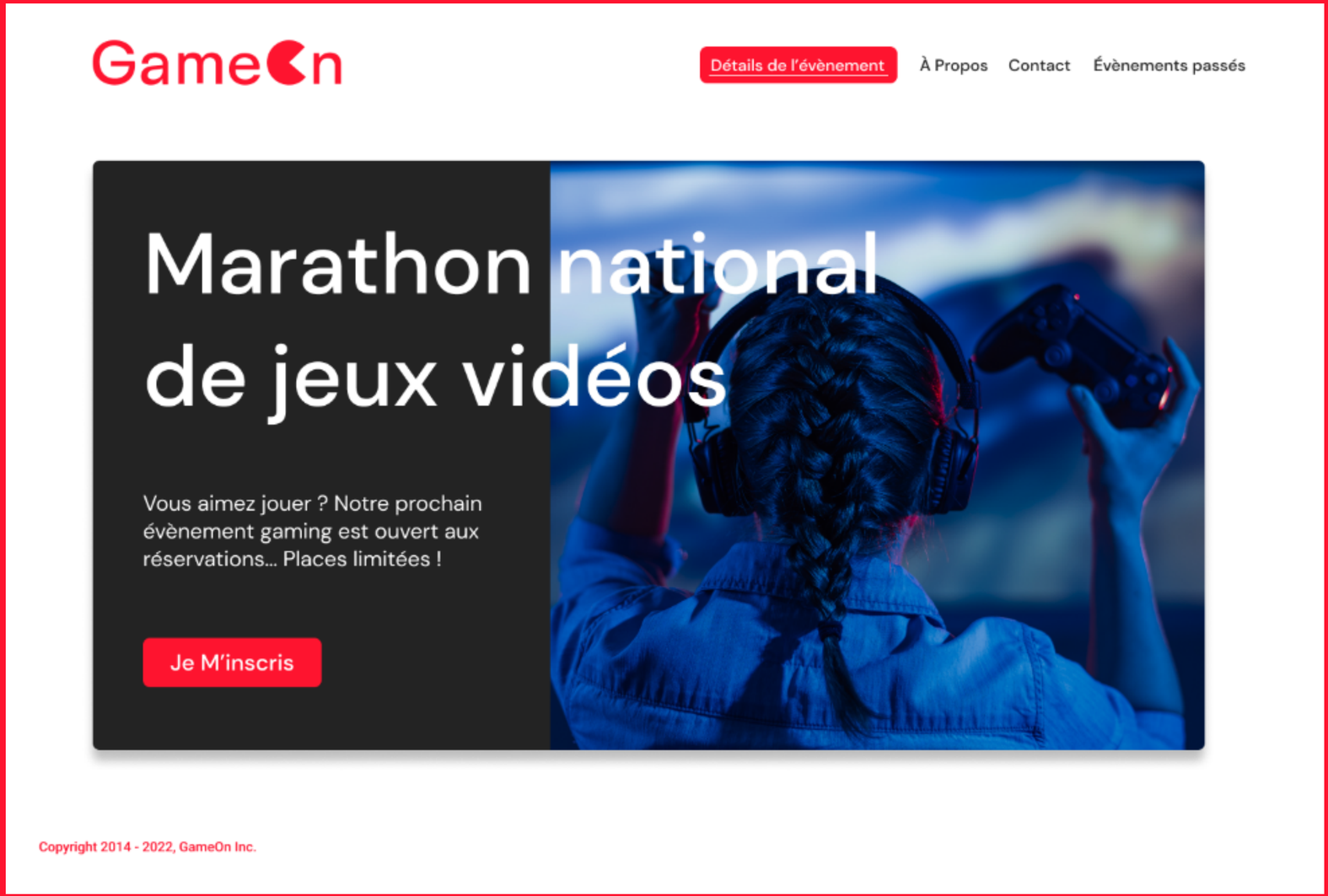
Quand tu auras terminé, on fera une révision du code tous les deux et on vérifiera qu'il est prêt à être mis en production.

Merci beaucoup,

Erika

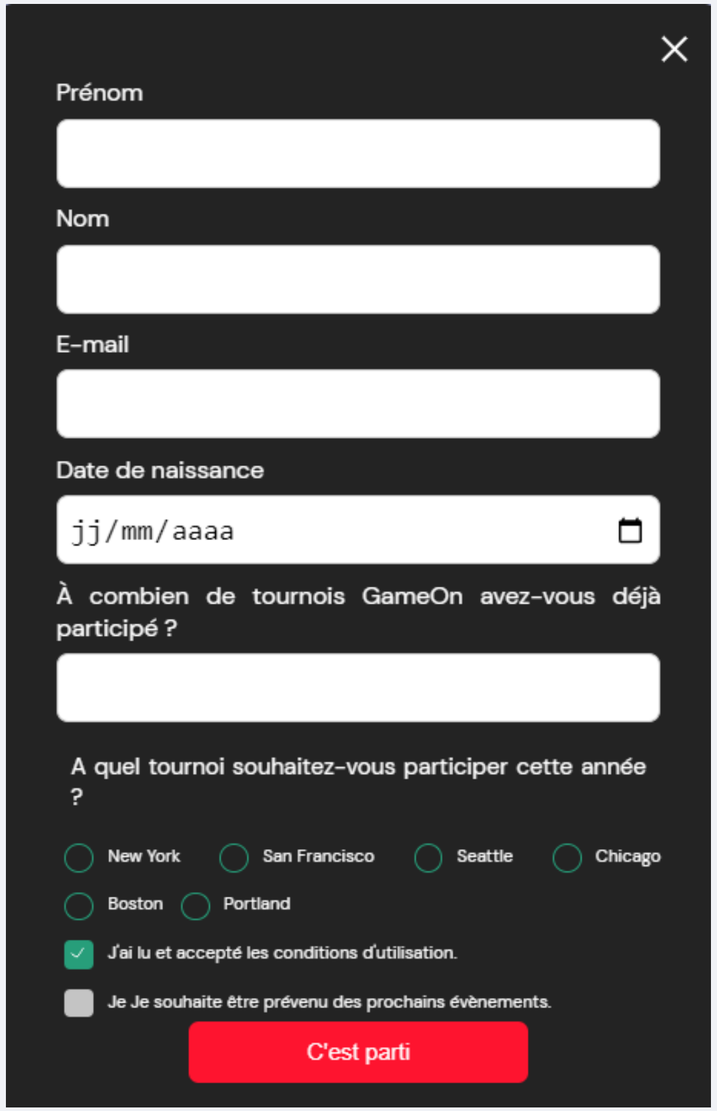
Présentation du projet

Vous ouvrez les maquettes pour avoir une idée de ce à quoi ressemble la page d'accueil :



FORMULAIRE

Notre travail consiste à ajouter le code JavaScript manquant pour que le formulaire soit pleinement fonctionnel.



Ce code JavaScript déclare des variables qui représentent différents éléments de la page HTML.

```
const modalbg = document.querySelector(".bground");
const modalBtn = document.querySelectorAll(".modal-btn");
const modalBtnClose = document.querySelector(".close");
const formData = document.querySelectorAll(".formData");
const form = document.getElementById("form");
const validForm = document.querySelector(".validationForm");
```

- La variable "modalbg" représente le fond du modal, qui est sélectionné à l'aide de la méthode "querySelector" en utilisant une sélecteur de classe CSS.
- La variable "modalBtn" représente un ensemble de boutons de modal, sélectionnés à l'aide de la méthode "querySelectorAll" avec un sélecteur de classe CSS.
- La variable "modalBtnClose" représente le bouton de fermeture du modal, sélectionné à l'aide de la méthode "querySelector" avec un sélecteur de classe CSS.
- La variable "formData" représente un ensemble de données de formulaire, sélectionnées à l'aide de la méthode "querySelectorAll" avec un sélecteur de classe CSS.
- La variable "form" représente le formulaire entier, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "form".
- La variable "validForm" représente la validation du formulaire, sélectionnée à l'aide de la méthode "querySelector" avec un sélecteur de classe CSS.

Ces variables seront utilisées plus tard dans le code pour interagir avec ces éléments HTML et manipuler leur comportement et leur apparence.

Ce code JavaScript déclare des variables qui représentent des éléments de formulaire spécifiques sur la page HTML.

```javascript
const firstName = document.getElementById("first");
const lastName = document.getElementById("last");
const email = document.getElementById("email");
const birthdate = document.getElementById("birthdate");
const quantity = document.getElementById("quantity");
const locationTournament = document.getElementsByName("location");
const condition = document.getElementById("checkbox1");
const validMessage = document.getElementById("validMessage");
const btnSubmit = document.getElementById("btnSubmit");
const btnValid = document.getElementById("btnValid");
```

Ce code JavaScript déclare des variables qui représentent des éléments de formulaire spécifiques sur la page HTML.

- La variable "firstName" représente le champ de formulaire pour le prénom, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "first".
- La variable "lastName" représente le champ de formulaire pour le nom de famille, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "last".
- La variable "email" représente le champ de formulaire pour l'adresse électronique, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "email".
- La variable "birthdate" représente le champ de formulaire pour la date de naissance, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "birthdate".
- La variable "quantity" représente le champ de formulaire pour la quantité, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "quantity".
- La variable "locationTournament" représente un ensemble de champs de formulaire pour la localisation du tournoi, sélectionnés à l'aide de la méthode "getElementsByName" avec le nom "location".
- La variable "condition" représente le champ de formulaire pour la condition, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "checkbox1".
- La variable "validMessage" représente le message de validation, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "validMessage".
- La variable "btnSubmit" représente le bouton de soumission de formulaire, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "btnSubmit".
- La variable "btnValid" représente le bouton de validation, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "btnValid".

Ces variables seront utilisées plus tard dans le code pour récupérer les valeurs des champs de formulaire et pour valider les entrées utilisateur.

Ce code JavaScript déclare des variables qui représentent des éléments de formulaire spécifiques sur la page HTML.

```
const firstText = document.getElementById("firstText");
const lastText = document.getElementById("lastText");
const emailText = document.getElementById("emailText");
const birthdateText = document.getElementById("birthdateText");
const quantityText = document.getElementById("quantityText");
const locationText = document.getElementById("locationText");
const conditionText = document.getElementById("conditionText");
```

Ce code JavaScript déclare des variables qui représentent des éléments de texte spécifiques sur la page HTML.

- La variable "firstText" représente un élément de texte pour le prénom, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "firstText".
- La variable "lastText" représente un élément de texte pour le nom de famille, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "lastText".
- La variable "emailText" représente un élément de texte pour l'adresse électronique, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "emailText".
- La variable "birthdateText" représente un élément de texte pour la date de naissance, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "birthdateText".
- La variable "quantityText" représente un élément de texte pour la quantité, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "quantityText".
- La variable "locationText" représente un élément de texte pour la localisation, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "locationText".
- La variable "conditionText" représente un élément de texte pour la condition, sélectionné à l'aide de la méthode "getElementById" avec l'identificateur d'ID "conditionText".

Ces variables seront utilisées plus tard dans le code pour afficher du texte à l'utilisateur pour indiquer l'état de la validation des entrées de formulaire.

Ces lignes de code définissent les éléments de la page HTML et les styles associés ainsi que les différents événements qui se produisent lorsqu'un utilisateur clique sur certaines parties de la page. La première partie de ce code définit des constantes qui réfèrent à différents éléments de la page HTML, tels que les boutons, les formulaires, les champs de formulaire, etc.
La seconde partie définit les événements qui se produisent lorsqu'un utilisateur clique sur certaines parties de la page. Par exemple, lorsqu'un utilisateur clique sur un bouton, la fonction "openModal" est déclenchée pour afficher le formulaire modal. De même, lorsqu'un utilisateur clique sur le bouton "Close", la fonction "hideModal" est déclenchée pour masquer le formulaire modal.
Enfin, la dernière partie du code définit une expression régulière (RegEx) pour valider les entrées utilisateur. L'expression régulière pour les noms et prénoms vérifie que l'entrée contient des lettres en majuscules et minuscules entre 2 à 20 et peut inclure un tiret. L'expression régulière pour les e-mails vérifie que l'entrée est dans le format d'une adresse e-mail valide, avec un format de nom d'utilisateur, un symbole @, un nom de domaine et une extension.

```
modalBtn.forEach((btn) => btn.addEventListener("click", ope



function openModal() {
  modalbg.style.display = "block";
};



function hideModal() {
  modalbg.style.display = "none";
};


modalBtnClose.addEventListener("click", function() {
  modalbg.style.display = "none";
});

// Création des RegEx
let regExTypeText = new RegExp(
  '^([A-Za-z]{2,20})?([-]{0,1})?([A-Za-z]{2,20})$'
);
let regExTypeEmail = new RegExp(
  '^[a-zA-Z0-9.-_]+[@]{1}[a-zA-Z0-9.-_]+[.]{1}[a-z]{2,10}$'
);
```

Ce code ajoute des événements de changement aux entrées du formulaire "firstName", "lastName" et "email". Lorsqu'une entrée change, la fonction "generiqueValidate" est appelée en passant en argument l'entrée elle-même, une expression régulière pour valider la saisie, un message d'erreur à afficher si la validation échoue et le champ de texte associé à cette entrée. La fonction "generiqueValidate" utilise la valeur de l'entrée et l'expression régulière pour valider la saisie et afficher un message d'erreur si nécessaire.

```
firstName.addEventListener('change', function() {
  generiqueValidate(this,regExTypeText,"Veuillez rentrer deux caractères minimum", firstText, this);
});

lastName.addEventListener('change', function() {
  generiqueValidate(this,regExTypeText,"Veuillez rentrer deux caractères minimum", lastText, this);
});

email.addEventListener('change', function() {
  generiqueValidate(this,regExTypeEmail,"Veuillez rentrer un adresse email valide", emailText, this);
});
```

Il semble que le code vérifie la valeur de l'entrée de la date de naissance et la valide. Si la valeur est vide ou non valide, il définit le message d'erreur et ajoute des classes CSS pour afficher l'état d'erreur. Si la valeur est valide, il définit un message de réussite et ajoute des classes CSS pour afficher l'état de réussite.

Cependant, la fonction validBirthdate ne prend pas d'argument, mais elle utilise la variable de date de naissance dans la fonction. Cette variable n'est pas déclarée dans la fonction et entraînera une ReferenceError. Pour résoudre ce problème, vous devez transmettre l'élément d'entrée de la date de naissance en tant qu'argument à la fonction et l'utiliser dans la fonction.

```
birthdate.addEventListener('change', function() {
  validBirthdate(this);
});

const validBirthdate = function() {

  if(! birthdate) {
    birthdateText.innerHTML = "Veuillez entrer une date de naissance valide";
    birthdateText.classList.remove('text-succes');
    birthdateText.classList.add('text-danger');
    birthdate.classList.remove('border-succes');
    birthdate.classList.add('border-danger');
    return false;
  }else {
    birthdateText.innerHTML = "Champs Valide";
    birthdateText.classList.remove('text-danger');
    birthdateText.classList.add('text-succes');
    birthdate.classList.remove('border-danger');
    birthdate.classList.add('border-succes');
    return true;
  }      You, il y a 24 heures • first commit
```

Le code ci-dessus est une fonction de validation pour un champ de formulaire appelé "quantité". La fonction ajoute un écouteur d'événement "change" à ce champ, ce qui signifie qu'elle sera appelée chaque fois que la valeur du champ change.

La fonction de validation en elle-même vérifie si la valeur saisie dans le champ "quantité" est égale à 0 ou inférieure à 0. Si tel est le cas, le message "Merci d'indiquer le nombre de tournois" sera affiché à côté du champ et la bordure du champ deviendra rouge pour indiquer une erreur de saisie. Si la valeur est supérieure à 50, le message "Nous n'avons pas organisé autant de tournois !" sera affiché et la bordure du champ deviendra également rouge. Si aucune de ces erreurs n'est détectée, le message "Champs valide" sera affiché et la bordure du champ deviendra verte pour indiquer une saisie valide.

```javascript
quantity.addEventListener('change', function() {
  validQuantity(this);
});

const validQuantity = function() {
  if(quantity.value === 0 || quantity.value < 0) {       // Si la valeur est égale à 0 ou si la valeur est supérieure à 0
    quantityText.innerHTML = "Merci d'indiquer le nombre de tournois";
    quantityText.classList.remove('text-succes');
    quantityText.classList.add('text-danger');
    quantity.classList.remove('border-succes');
    quantity.classList.add('border-danger');
    return false;
  }else if(quantity.value > 50) {
    quantityText.innerHTML = "Nous n'avons pas organisé autant de tournois !";
    quantityText.classList.remove('text-succes');
    quantityText.classList.add('text-danger');
    quantity.classList.remove('border-succes');
    quantity.classList.add('border-danger');
    return false;
  }else {
    quantityText.innerHTML = "Champs Valide";
    quantityText.classList.remove('text-danger');
    quantityText.classList.add('text-succes');
    quantity.classList.remove('border-danger');
    quantity.classList.add('border-succes');      You, hier • first commit …
    return true;
  }
};
```

En somme, cette fonction vise à informer l'utilisateur s'il a entré une quantité valide ou non.

Ce code JavaScript fait partie d'un formulaire d'inscription. Il comporte plusieurs parties distinctes qui vérifient les informations entrées par l'utilisateur et affichent un message de validation ou d'erreur selon les cas.

1. La première partie concerne la vérification des conditions d'utilisation :
- Lorsque l'utilisateur clique sur le champ "condition", une fonction "validCondition" est appelée pour vérifier si la case à cocher a été cochée ou non.
- Si la case n'a pas été cochée, le message "Merci d'accepter les conditions d'utilisations" s'affiche.
- Sinon, le message "Champs Valide" est affiché.
2. La deuxième partie concerne la validation du formulaire :
- La fonction "validate" appelle plusieurs autres fonctions qui vérifient chacun un champ du formulaire (prénom, nom, email, date de naissance, quantité de tournois et ville de tournois).
- Si toutes les vérifications retournent "true", la fonction "openRemerciments" est appelée pour afficher un message de remerciement pour l'inscription.
- Si une vérification retourne "false", un message d'alerte "Merci de remplir correctement votre inscription" s'affiche.
3. La troisième partie concerne le bouton "soumettre" :
- Lorsque l'utilisateur clique sur le bouton "soumettre", la fonction "validate" est appelée pour vérifier tous les champs du formulaire.
- Si tous les champs sont correctement remplis, la page se recharge.

```javascript
condition.addEventListener('change', function() {
  validCondition(this);
});

// Vérifie si les conditions sont biens cochées ou non
const validCondition = function() {
  if(condition.checked == false ) {
    conditionText.innerHTML = "Merci d'accepter les conditions d'utilisations";
    conditionText.classList.remove('text-succes');
    conditionText.classList.add('text-danger');
    return false;
  }else {
    conditionText.innerHTML = "Champs Valide";
    conditionText.classList.remove('text-danger');
    conditionText.classList.add('text-succes');
    return true;
  }
};

//----- BTN VALIDATION -----
function openRemerciments() {
  form.style.display = "none";
  validForm.style.display = "flex";
  validMessage.innerHTML = "Merci pour votre inscription";
};
```

```javascript
function validate() {
  // Condition qui vérifie si tous les autres conditions retourne true
  if (generiqueValidate(firstName, regExTypeText, "firstname error", firstText, firstName)
  && generiqueValidate(lastName, regExTypeText, "lastname error", lastText, lastName)
  && generiqueValidate(email, regExTypeEmail, "email error", emailText, email)
  && validBirthdate(birthdate)
  && validQuantity(quantity)
  && validLocationTournament()
  && validCondition(condition)) {

    openRemerciments();

  }else {
    alert("Merci de remplir correctement votre inscription");
  }
}

//----- BTN SUBMIT -----

btnValid.addEventListener("click", function() {
  window.location.reload();
});
```

Fin de la Soutenance Projet OpenClassrooms

GameOn

Créez une landing page avec Javascript

Présenter par Ghani